

Software Requirements Specification (SRS)

Project: "Wafeer" Clone – Cross-Platform SMS Expense Tracker

1. Project Overview

Goal: Develop a mobile application that automatically detects incoming bank SMS messages, parses the transaction details (Amount, Merchant, Date), categorizes the expense, and visualizes the user's spending habits.

Target Platforms:

- **Android:** Native background SMS interception (Zero-touch).
- **iOS:** Semi-automated ingestion via Apple Shortcuts & Deep Linking.
Privacy Policy: Offline First. All data processing and storage must occur locally on the device. No financial data is to be sent to external servers.

2. Functional Requirements (FR)

FR-1: Android Background Listener (Native)

- **Requirement:** The app must register a `BroadcastReceiver` to listen for `android.provider.Telephony.SMS_RECEIVED`.
- **Constraint:** The listener must filter messages using a `BankWhitelist` (e.g., Sender IDs like "AIRajhi", "SNB", "STCPay") to ignore personal texts or OTPs.
- **Trigger:** Arrival of a valid SMS triggers the Parsing Engine in the background.

FR-2: iOS Deep Link Handler (The Bridge)

- **Requirement:** The app must register a custom URL scheme (e.g., `wafeerclone://`) to accept data from external sources.
- **Workflow:**
 1. **User Action:** User receives SMS $\rightarrow$ Apple Shortcut automation runs $\rightarrow$ User taps "Run".
 2. **Data Transfer:** Shortcut opens URL: `wafeerclone://log?s=shortcut&b=[URL_ENCODED_SMS_BODY]`.
 3. **App Response:** App wakes up, decodes the body, runs the Parsing Engine, and saves the transaction.

FR-3: The Regex Parsing Engine

- **Input:** Raw SMS String.
- **Logic:**
 1. **Keywords:** Check for "Purchase", "Debit", "Withdrawal".

2. **Amount:** Extract numerical value following currency codes (SAR, USD, AED).
3. **Merchant:** Capture text between "at"/"from" and "on"/"date".
4. **Date:** Parse standard formats (DD-MM-YYYY, YYYY-MM-DD).

FR-4: Auto-Categorization

- **Logic:** Map **Merchant Name** to **Category** using a keyword dictionary.
 - *Examples:* "Burger" $\rightarrow$ Food; "Station" $\rightarrow$ Transport; "Panda" $\rightarrow$ Groceries.
 - *Default:* If no match, assign "Uncategorized".

FR-5: User Dashboard & UI

- **Home:** Display "Total Spent This Month" and a list of recent transactions.
- **Manual Entry:** "Paste from Clipboard" button for older SMS messages.
- **Onboarding:**
 - **Android:** Request **RECEIVE_SMS** and **READ_SMS** permissions.
 - **iOS:** Tutorial screen linking to the required Apple Shortcut download.

3. Technical Requirements (TR)

TR-1: System Architecture

- **Framework:** Flutter (Dart 3.0+).
- **Architecture Pattern:** MVVM or Clean Architecture.
- **Data Layer:** Local Database (NoSQL) with repository pattern.

TR-2: Technology Stack & Packages

- **SMS Handling (Android):** **telephony** (Supports background isolates).
- **Database:** **isar** (Preferred for speed & encryption) or **drift**.
- **Deep Linking:** **app_links** or **uni_links** (For iOS Shortcuts integration).
- **State Management:** **flutter_riverpod** or **provider**.
- **Permissions:** **permission_handler**.

TR-3: Data Security

- **Encryption:** The local database must be encrypted using AES-256. The key should be generated on install and stored in **Flutter Secure Storage** (KeyStore/Keychain).
- **Network:** The app should not require **INTERNET** permission in its final release build to ensure data privacy (unless cloud backup is explicitly added later).

TR-4: Platform Configuration

Android Manifest (**AndroidManifest.xml**):

XML

```
<uses-permission android:name="android.permission.RECEIVE_SMS" /> <uses-permission  
android:name="android.permission.READ_SMS" /> <uses-permission  
android:name="android.permission.FOREGROUND_SERVICE" />
```

iOS Configuration (**Info.plist**):
XML

```
<key>CFBundleURLTypes</key> <array> <dict> <key>CFBundleURLName</key>  
<string>com.example.wafeerclone</string> <key>CFBundleURLSchemes</key> <array>  
<string>wafeerclone</string> </array> </dict> </array><key>CFBundleURLTypes</key>
```

TR-5: Data Models (JSON Schema)

Transaction Entity:

JSON

```
{ "id": "UUID", "source": "Enum [ANDROID_BG, IOS_SHORTCUT, MANUAL]", "bankName":  
"String", "merchant": "String", "amount": "Double", "currency": "String (Default: SAR)",  
"category": "String", "timestamp": "DateTime", "rawBody": "String" }
```

Category Rule Entity:

JSON

```
{ "id": "UUID", "name": "String", "keywords": ["String"], "iconAsset": "String" }
```

4. Regex Implementation Guide

Pattern A: Purchase (POS)

- **Regex:**
`(?:Purchase|Debit).*?(?:SAR|AED)\s*(\d+(?:\.\d{1,2})?).*?at\s+(.*?)\s+on`
- **Target:** Group 1 = Amount, Group 2 = Merchant.

Pattern B: Cash Withdrawal

- **Regex:** `(?:Cash Withdrawal).*?(?:SAR|AED)\s*(\d+(?:\.\d{1,2})?)`
- **Target:** Group 1 = Amount. (Set Merchant to "Cash Withdrawal").